

Copart Conversational Hybrid Search System

Structured SQL + Semantic Inspector Notes RAG & Knowledge Base Documentation

Executive Summary: Implements an enterprise-grade **Hybrid Conversational Search** platform for Copart vehicle buyers. It marries **Deterministic SQL Filters** (for exact bounds: price, make, year, location) with **Semantic Vector/FTS5 RAG** (for unstructured mechanic inspection notes like *'airbags intact'*, *'easy DIY fix'*, *'light hail for PDR'*) and an integrated **Copart Policy Knowledge Base** (explaining salvage titles, broker bidding, and condition codes).
Stack: React + Vite (Frontend) • FastAPI Python (Backend) • SQLite + FTS5 BM25 (Hybrid Database) • Gemini 3.6 Flash / OpenAI Tool Calling.

1. How Hybrid Search Works (Real-World Analogy)

Standard search engines make you choose between rigid dropdowns or loose keyword search. Our Hybrid Search gives buyers both simultaneously:

- **Structured Query:** *'Toyota SUVs in Texas under \$15k'* → Database uses B-Tree indexes to strictly filter matching cars in <2ms.
- **Semantic / Inspector RAG:** *'Easy to fix with no airbag damage'* → RAG searches unstructured mechanic notes to find low-severity collision lots.
- **Copart Policy RAG:** *'What does Enhanced Vehicle mean?'* → Retrieves official Copart definitions and guidance inside the chat.
- **Multi-Turn Memory:** *'Actually make it Honda'* → Swaps Make to Honda while keeping price, state, and inspection criteria intact.

2. End-to-End Hybrid Search Workflow

Step	Layer	What Happens Behind the Scenes
1	React UI (Port 5173)	User enters query (e.g. <i>'Toyota SUVs easy to fix with no airbag damage'</i>). Dispatches POST /chat payload.
2	FastAPI Gateway (Port 8000)	main.py pulls session state, loads history from session.py, and injects Copart domain instructions.
3	LLM Agent & RAG (agent.py)	Extracts structured params (make: Toyota) AND semantic query (semantic_query: 'easy fix DIY airbags intact'). Queries policy knowledge base.
4	Session State (session.py)	Merges newly extracted parameters into active session filters (preserving prior turn filters unless overridden).
5	Hybrid DB Engine (inventory.py)	Runs combined SQL query: hard SQL WHERE clauses for price/make/state + FTS5 BM25 ranking on inspector_notes.
6	Live UI Update	React renders conversational response, Copart policy guide tips, active filter tags, and vehicle cards with technician note excerpts.

3. Deep Dive: Backend Codebase & Architecture

All backend source code lives inside `backend/`. Below is the exact technical function of every module:

A. `main.py` — The API Gateway & Traffic Controller

Role: Central FastAPI router coordinating the conversational AI pipeline.

- **POST /chat:** Receives user message, runs agent, merges session state, queries hybrid DB, and returns dialogue + vehicles + policy guidance.
- **GET /vehicles:** Direct programmatic vehicle search with filter and semantic query parameters (e.g. `?make=Toyota&semantic;_query=hail`).
- **GET /filter-options & GET /health:** Dynamic UI filter bounds and backend liveness verification.

B. `agent.py` — Hybrid Tool-Calling & Knowledge RAG

Role: Translates human language into structured filters, semantic inspector queries, and policy answers.

- **Hybrid Tool Schema:** Formal `search_vehicles` schema accepting `make`, `price_max`, `location_state`, and `semantic_query`.
- **Multi-Model Flexibility:** Connects to Gemini 3.6 Flash (via Google Generative Language API) or OpenAI (via `openai`).
- **Zero-Cost Fallback Engine:** Includes a built-in NLP pattern extractor + SQLite FTS5 RAG matcher for 100% testability without API keys.

C. `inventory.py` — Hybrid SQL + BM25 Semantic Query Layer

Role: Executes safe, high-performance database lookups.

- **Hybrid Ranking:** Applies strict SQL filters (`price <= 15000`) while ranking results by BM25 semantic match score on `inspector_notes`.
- **Policy Knowledge RAG (`query_knowledge_base`):** Performs instant retrieval on Copart FAQs, title regulations, and auction rules.
- **SQL Injection Safe:** Always uses parameterized SQL placeholders (`?`).

D. `session.py` — Multi-Turn Memory & State Machine

Role: Manages multi-turn conversation memory and filter persistence.

- **Filter Accumulation:** Accumulates constraints across turns (e.g. Turn 1: *Toyota* + Turn 2: *under \$15k* → *[Toyota, <\$15k]*).
- **Filter Overriding:** Replaces values on demand (e.g. *'actually show Honda'* replaces *make* while preserving other filters).
- **Sliding History Window:** Truncates message history to the last 20 turns to conserve LLM token budget.

E. `seed_data.py` & `models.py` — Data Generation & Validation

- **`seed_data.py`:** Generates 500 realistic vehicles with authentic VINs, lot numbers, and detailed mechanic inspector notes.
- **`models.py`:** Pydantic schemas (`Vehicle`, `VehicleFilters`, `ChatRequest`, `ChatResponse`) ensuring strict type safety.

4. Deep Dive: Frontend Components & UI System

All frontend code resides in frontend/src/, built with React 18, Vite, and custom CSS:

Component / File	Visual & Functional Role
App.jsx	Root layout coordinator. Manages global state (vehicle result list, active filter object, total matches) and seamlessly links the left Chat Panel with the right Results Grid.
ChatInterface.jsx	Interactive dialogue sidebar with hybrid search suggestion chips (e.g. <i>'Toyota SUVs easy to fix with no airbag damage'</i>), auto-expanding textarea, typing animations, and reset button.
VehicleCard.jsx	Vehicle card displaying vehicle photo, price, year/make/model, lot number, location, mileage, transmission, damage type, color-coded condition badge, and a dedicated Inspector Note Excerpt .
FilterPanel.jsx	Active filters bar displaying extracted structured filters (e.g. [Make: Toyota]) and semantic search tags (e.g. [Inspector Search: easy fix DIY]).
ConversationBubble.jsx	Renders user message bubbles (blue gradient, right aligned) and assistant bubbles (dark slate, left aligned) with timestamps and animated typing dots.
index.css	Comprehensive dark-mode design system with glassmorphism cards, smooth hover micro-animations, glowing focus borders, inspector note boxes, and responsive layouts.
vite.config.js	Configures lightning-fast local development with Hot Module Replacement (HMR) and an automatic proxy routing /chat and /vehicles requests directly to FastAPI on port 8000.

5. Security, Guardrails & Reliability Architecture

Engineering & Reliability Principles:

- **SQL Injection Prevention:** All database queries compile strictly via parameterized placeholders (?), preventing arbitrary SQL execution regardless of user query contents.
- **Strict Type Safety:** Pydantic v2 data models enforce validation on all inbound chat payloads, search filters, and outgoing vehicle records.
- **Provider Resilience & Graceful Degradation:** Pluggable LLM architecture routes seamlessly between Gemini and OpenAI, with automatic fallback to local rule-based parsing if network or quota issues occur.
- **Bounded Memory:** Multi-turn conversation sessions maintain an active sliding window to prevent unbounded memory growth and optimize context consumption.

6. Step-by-Step Testing & Verification Commands

Run the commands below in your terminal to launch and test the entire stack:

Step 1: Start Backend API (Terminal 1)

```
cd "/Users/bhargav/Downloads/copart project/backend"
source venv/bin/activate
uvicorn main:app --reload --port 8000
```

Step 2: Start Frontend Application (Terminal 2)

```
cd "/Users/bhargav/Downloads/copart project/frontend"
npm run dev
```

Open **<http://localhost:5173>** in your browser to interact with the UI.

Step 3: Test Hybrid Search & Policy RAG via cURL (Terminal 3)

```
# 1. Health check
curl http://localhost:8000/health

# 2. Hybrid Search Test: Structured SQL + Semantic Inspector Notes
curl -s -X POST http://localhost:8000/chat \
-H "Content-Type: application/json" \
-d '{"message": "Show me Toyota SUVs easy to fix with no airbag damage"}' | python3 -m json.tool

# 3. Policy Knowledge RAG Test: Copart Guidelines
curl -s -X POST http://localhost:8000/chat \
-H "Content-Type: application/json" \
-d '{"message": "What does Enhanced Vehicle mean at Copart?"}' | python3 -m json.tool

# 4. Multi-Turn Search Refinement Test
SESSION=$(curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" \
-d '{"message": "Show me Ford trucks"}' | python3 -c "import sys,json; print(json.load(sys.stdin)['session_id'])")

curl -s -X POST http://localhost:8000/chat -H "Content-Type: application/json" \
-d '{"message": "Under $15000", "session_id": "$SESSION"}' | python3 -m json.tool
```

7. Evolution to Production & Enterprise Scaling

- **Database & Scale:** Migrate SQLite to PostgreSQL with pgvector or OpenSearch for hybrid semantic (embedding) and faceted search across millions of salvage inventory lots.
- **Distributed Session Store:** Transition in-memory session dictionary to Redis with TTL expiration for horizontal auto-scaling across container replicas.
- **Real-Time Inventory CDC:** Ingest continuous lot updates (bids, yard arrivals, title changes) via Kafka / AWS Kinesis change-data-capture pipelines.
- **Production Guardrails:** Add prompt injection firewalls (NeMo Guardrails), rate limiting via API Gateway, and latency tracking using LangSmith / OpenTelemetry.